

# Sistemas Operativos

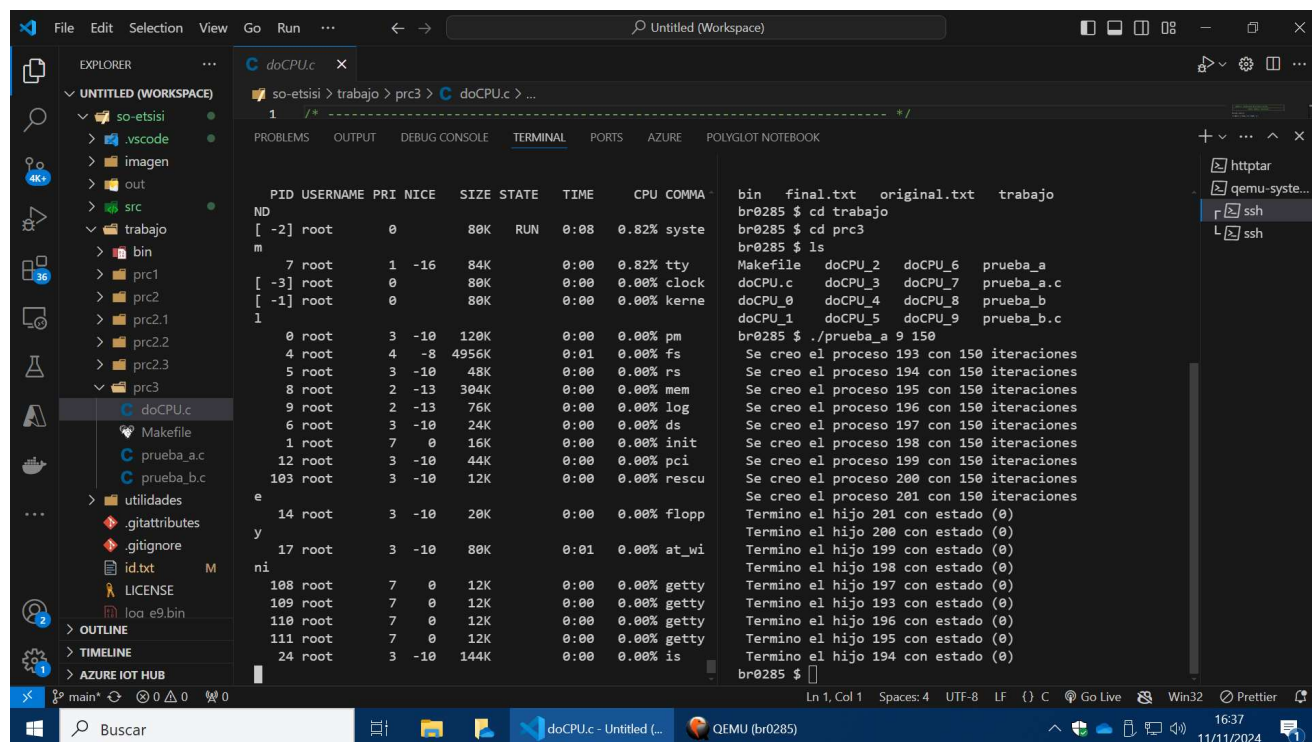
## Práctica 3

Fecha: 11/11/2024

Alumno: Yzan Martinez Manzano

Correo: [yzan.martinez.manzano@alumnos.upm.es](mailto:yzan.martinez.manzano@alumnos.upm.es)

## 1. Prueba a 9



The screenshot shows a VS Code editor with a terminal window open. The terminal displays a process list and the execution of a script named `prueba_a`.

**Process List:**

| PID   | USERNAME | PRI | NICE | SIZE  | STATE | TIME | CPU   | COMMA |
|-------|----------|-----|------|-------|-------|------|-------|-------|
| ND    |          |     |      |       |       |      |       |       |
| [ -2] | root     | 0   |      | 80K   | RUN   | 0:08 | 0.82% | syste |
| 7     | root     | 1   | -16  | 84K   |       | 0:00 | 0.82% | tty   |
| [ -3] | root     | 0   |      | 80K   |       | 0:00 | 0.00% | clock |
| [ -1] | root     | 0   |      | 80K   |       | 0:00 | 0.00% | kerne |
| 1     | root     | 3   | -10  | 120K  |       | 0:00 | 0.00% | pm    |
| 4     | root     | 4   | -8   | 4956K |       | 0:01 | 0.00% | fs    |
| 5     | root     | 3   | -10  | 48K   |       | 0:00 | 0.00% | rs    |
| 8     | root     | 2   | -13  | 304K  |       | 0:00 | 0.00% | mem   |
| 9     | root     | 2   | -13  | 76K   |       | 0:00 | 0.00% | log   |
| 6     | root     | 3   | -10  | 24K   |       | 0:00 | 0.00% | ds    |
| 1     | root     | 7   |      | 16K   |       | 0:00 | 0.00% | init  |
| 12    | root     | 3   | -10  | 44K   |       | 0:00 | 0.00% | pci   |
| 103   | root     | 3   | -10  | 12K   |       | 0:00 | 0.00% | rescu |
| e     |          |     |      |       |       |      |       |       |
| 14    | root     | 3   | -10  | 20K   |       | 0:00 | 0.00% | flopp |
| y     |          |     |      |       |       |      |       |       |
| 17    | root     | 3   | -10  | 80K   |       | 0:01 | 0.00% | at_wi |
| ni    |          |     |      |       |       |      |       |       |
| 108   | root     | 7   |      | 12K   |       | 0:00 | 0.00% | getty |
| 109   | root     | 7   |      | 12K   |       | 0:00 | 0.00% | getty |
| 110   | root     | 7   |      | 12K   |       | 0:00 | 0.00% | getty |
| 111   | root     | 7   |      | 12K   |       | 0:00 | 0.00% | getty |
| 24    | root     | 3   | -10  | 144K  |       | 0:00 | 0.00% | is    |

**Script Execution:**

```
bin final.txt original.txt trabajo
br0285 $ cd trabajo
br0285 $ cd prc3
br0285 $ ls
Makefile doCPU_2 doCPU_6 prueba_a
doCPU.c doCPU_3 doCPU_7 prueba_a.c
doCPU_0 doCPU_4 doCPU_8 prueba_b
doCPU_1 doCPU_5 doCPU_9 prueba_b.c
br0285 $ ./prueba_a 9 150
Se creo el proceso 193 con 150 iteraciones
Se creo el proceso 194 con 150 iteraciones
Se creo el proceso 195 con 150 iteraciones
Se creo el proceso 196 con 150 iteraciones
Se creo el proceso 197 con 150 iteraciones
Se creo el proceso 198 con 150 iteraciones
Se creo el proceso 199 con 150 iteraciones
Se creo el proceso 200 con 150 iteraciones
Se creo el proceso 201 con 150 iteraciones
Termino el hijo 201 con estado (0)
Termino el hijo 200 con estado (0)
Termino el hijo 199 con estado (0)
Termino el hijo 198 con estado (0)
Termino el hijo 197 con estado (0)
Termino el hijo 193 con estado (0)
Termino el hijo 196 con estado (0)
Termino el hijo 195 con estado (0)
Termino el hijo 194 con estado (0)
br0285 $
```

The screenshot shows a VS Code editor with a terminal window open. The terminal displays system statistics and process creation commands. The statistics show load averages, memory usage, and CPU states. The commands create processes with names like doCPU\_0 through doCPU\_9. The terminal output shows the creation of processes 193 through 201, each with 150 iterations, and their subsequent termination.

```
load averages: 0.04, 0.04, 0.61
42 processes: 1 running, 41 sleeping
Mem: 490432K Free, 490236K Contiguous Free
CPU states: 0.00% user, 0.82% system, 0.82% kernel, 98.36% idle

PID USERNAME PRI NICE SIZE STATE TIME CPU COMMA
load averages: 0.04, 0.04, 0.61
42 processes: 1 running, 41 sleeping
Mem: 490432K Free, 490236K Contiguous Free
CPU states: 0.00% user, 0.82% system, 0.82% kernel, 98.36% idle

PID USERNAME PRI NICE SIZE STATE TIME CPU COMMA
load averages: 0.04, 0.04, 0.61
42 processes: 1 running, 41 sleeping
Mem: 490432K Free, 490236K Contiguous Free
CPU states: 0.00% user, 0.82% system, 0.82% kernel, 98.36% idle

PID USERNAME PRI NICE SIZE STATE TIME CPU COMMA
load averages: 0.04, 0.04, 0.61
42 processes: 1 running, 41 sleeping
Mem: 490432K Free, 490236K Contiguous Free
CPU states: 0.00% user, 0.82% system, 0.82% kernel, 98.36% idle

bin final.txt original.txt trabajo
br0285 $ cd trabajo
br0285 $ cd prc3
br0285 $ ls
Makefile doCPU_2 doCPU_6 prueba_a
doCPU.c doCPU_3 doCPU_7 prueba_a.c
doCPU_0 doCPU_4 doCPU_8 prueba_b
doCPU_1 doCPU_5 doCPU_9 prueba_b.c
br0285 $ ./prueba_a 9 150
Se creo el proceso 193 con 150 iteraciones
Se creo el proceso 194 con 150 iteraciones
Se creo el proceso 195 con 150 iteraciones
Se creo el proceso 196 con 150 iteraciones
Se creo el proceso 197 con 150 iteraciones
Se creo el proceso 198 con 150 iteraciones
Se creo el proceso 199 con 150 iteraciones
Se creo el proceso 200 con 150 iteraciones
Se creo el proceso 201 con 150 iteraciones
Termino el hijo 201 con estado (0)
Termino el hijo 200 con estado (0)
Termino el hijo 199 con estado (0)
Termino el hijo 198 con estado (0)
Termino el hijo 197 con estado (0)
Termino el hijo 193 con estado (0)
Termino el hijo 196 con estado (0)
Termino el hijo 195 con estado (0)
Termino el hijo 194 con estado (0)
br0285 $
```

¿Por qué la prioridad de los procesos doCPU\_x va disminuyendo progresivamente desde el momento en el que se crean?

La prioridad de los procesos doCPU\_x baja poco a poco desde el momento en que se crean porque Minix reduce la prioridad de los procesos que usan mucha CPU. Esto evita que un solo proceso ocupe la CPU todo el tiempo y permite que otros procesos también tengan oportunidad de ejecutarse.

¿Por qué a veces se incrementa?

A veces, la prioridad de estos procesos sube gracias a la función balance\_queue(), que ajusta las prioridades cada cierto tiempo. Esto permite que los procesos que han estado ocupados con tareas intensas puedan recuperar su prioridad inicial y seguir ejecutándose sin quedarse siempre en la prioridad más baja.

## 2. Prueba a 36

```
so-etsisi > trabajo > prc3 > C doCPU.c > main(int, char * [])
1 /*
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS AZURE POLYGLOT NOTEBOOK
U_8      142 usuario  9   0  24K  RUN  0:01  6.56% doCP
U_0      143 usuario  9   0  24K  RUN  0:01  6.56% doCP
U_1      144 usuario  9   0  24K  RUN  0:01  6.56% doCP
U_2      145 usuario  9   0  24K  RUN  0:01  6.56% doCP
U_3      146 usuario  9   0  24K  RUN  0:01  6.56% doCP
U_4      147 usuario  9   0  24K  RUN  0:01  6.56% doCP
U_5      148 usuario  9   0  24K  RUN  0:01  6.56% doCP
U_6      167 usuario 10   0  24K  RUN  0:00  6.56% doCP
U_7      149 usuario  9   0  24K  RUN  0:01  5.74% doCP
U_4      4 root      4  -8 495K  0:00  0.82% fs
U_5      58 root      3 -10 780K  0:00  0.82% inet
U_6      166 usuario  9   0  24K  RUN  0:00  0.82% doCP
U_4      [-3] root      0   0  80K   0:00  0.00% cloc
U_5      k          0   0  80K   0:05  0.00% syst
U_6      em
Se creo el proceso 140 con 150 iteraciones
Se creo el proceso 141 con 150 iteraciones
Se creo el proceso 142 con 150 iteraciones
Se creo el proceso 143 con 150 iteraciones
Se creo el proceso 144 con 150 iteraciones
Se creo el proceso 145 con 150 iteraciones
Se creo el proceso 146 con 150 iteraciones
Se creo el proceso 147 con 150 iteraciones
Se creo el proceso 148 con 150 iteraciones
Se creo el proceso 149 con 150 iteraciones
Se creo el proceso 150 con 150 iteraciones
Se creo el proceso 151 con 150 iteraciones
Se creo el proceso 152 con 150 iteraciones
Se creo el proceso 153 con 150 iteraciones
Se creo el proceso 154 con 150 iteraciones
Se creo el proceso 155 con 150 iteraciones
Se creo el proceso 156 con 150 iteraciones
Se creo el proceso 157 con 150 iteraciones
Se creo el proceso 158 con 150 iteraciones
Se creo el proceso 159 con 150 iteraciones
Se creo el proceso 160 con 150 iteraciones
Se creo el proceso 161 con 150 iteraciones
Se creo el proceso 162 con 150 iteraciones
Se creo el proceso 163 con 150 iteraciones
Se creo el proceso 164 con 150 iteraciones
Se creo el proceso 165 con 150 iteraciones
Se creo el proceso 166 con 150 iteraciones
Se creo el proceso 167 con 150 iteraciones
```

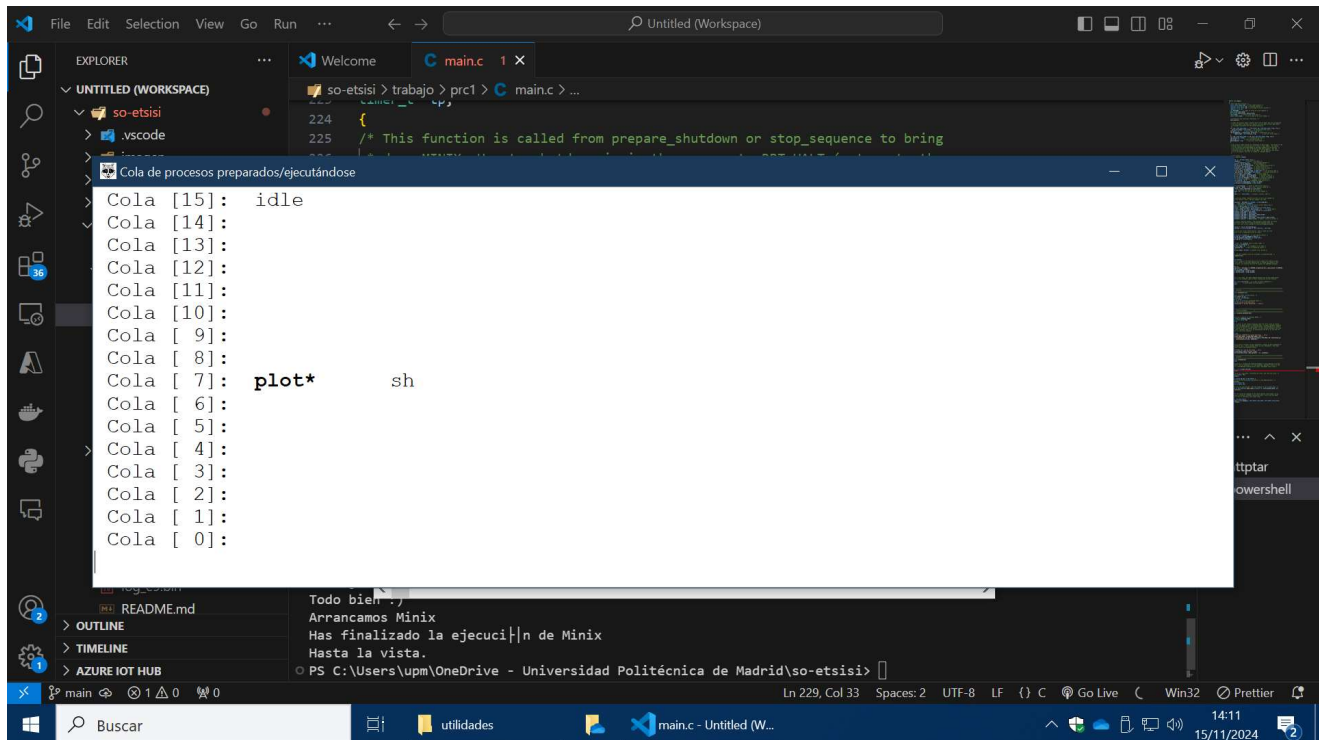
-¿En qué se diferencia el comportamiento de la prioridad de los procesos doCPU\_x con respecto a la prueba anterior?

En la prueba con 36 procesos doCPU\_x, la prioridad de estos procesos baja más rápido que en la prueba de 9 procesos. Esto pasa porque hay más procesos compitiendo por la CPU, y cada uno recibe menos tiempo, lo que hace que su prioridad baje rápidamente.

- ¿Por qué ocurre esto? Explícalo teniendo en cuenta el papel que juegan las funciones `balance_queues()` y `sched()`.

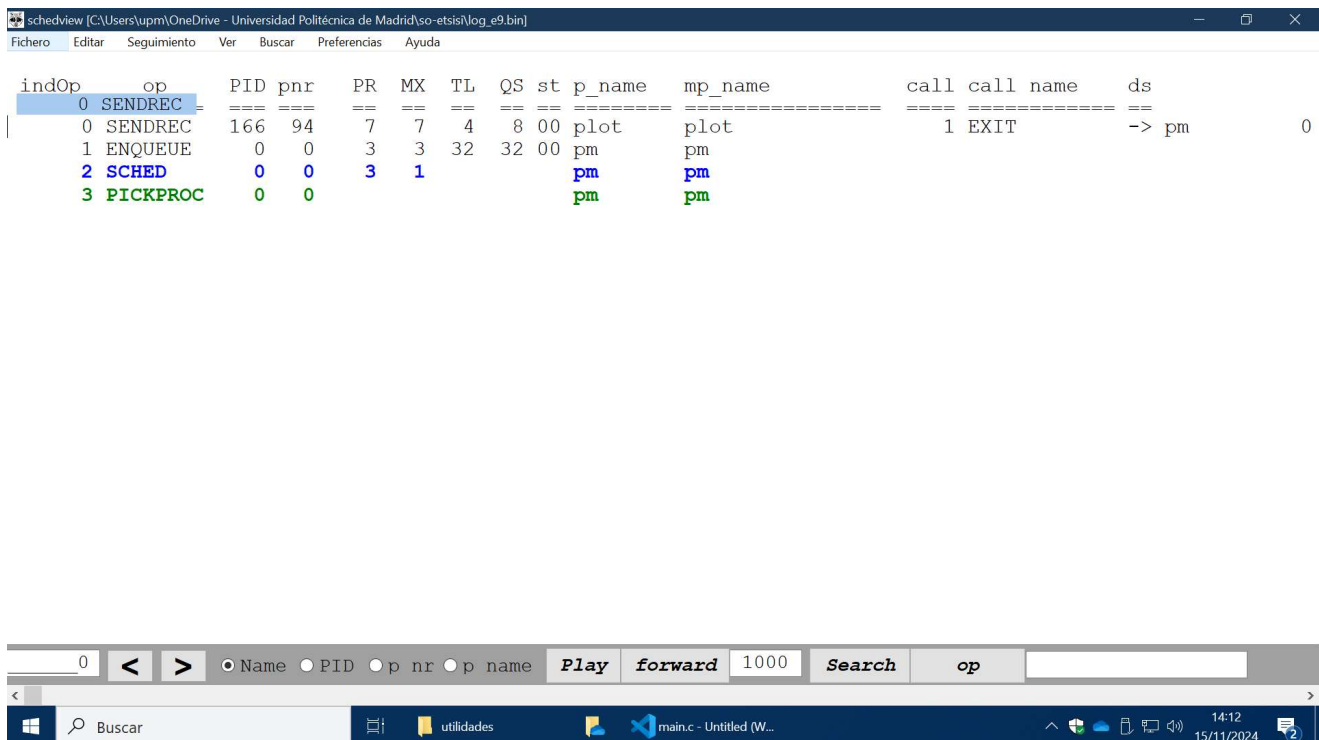
Cuando un proceso usa todo su tiempo de CPU, `sched()` le baja la prioridad para darles oportunidad a otros procesos. Y por último la función `balance_queues()` ajusta las prioridades de los procesos cada cierto tiempo, subiéndolas si han estado mucho tiempo con prioridad baja. Sin embargo, con 36 procesos, esta función no puede subir las prioridades lo suficiente para todos, por lo que la mayoría queda en colas de prioridad baja debido al alto consumo de CPU.

### 3. Modificando prioridades con nice



Cola de procesos preparados/ejecutándose

```
Cola [15]: idle
Cola [14]:
Cola [13]:
Cola [12]:
Cola [11]:
Cola [10]:
Cola [ 9]:
Cola [ 8]:
Cola [ 7]: plot*      sh
Cola [ 6]:
Cola [ 5]:
Cola [ 4]:
Cola [ 3]:
Cola [ 2]:
Cola [ 1]:
Cola [ 0]:
```



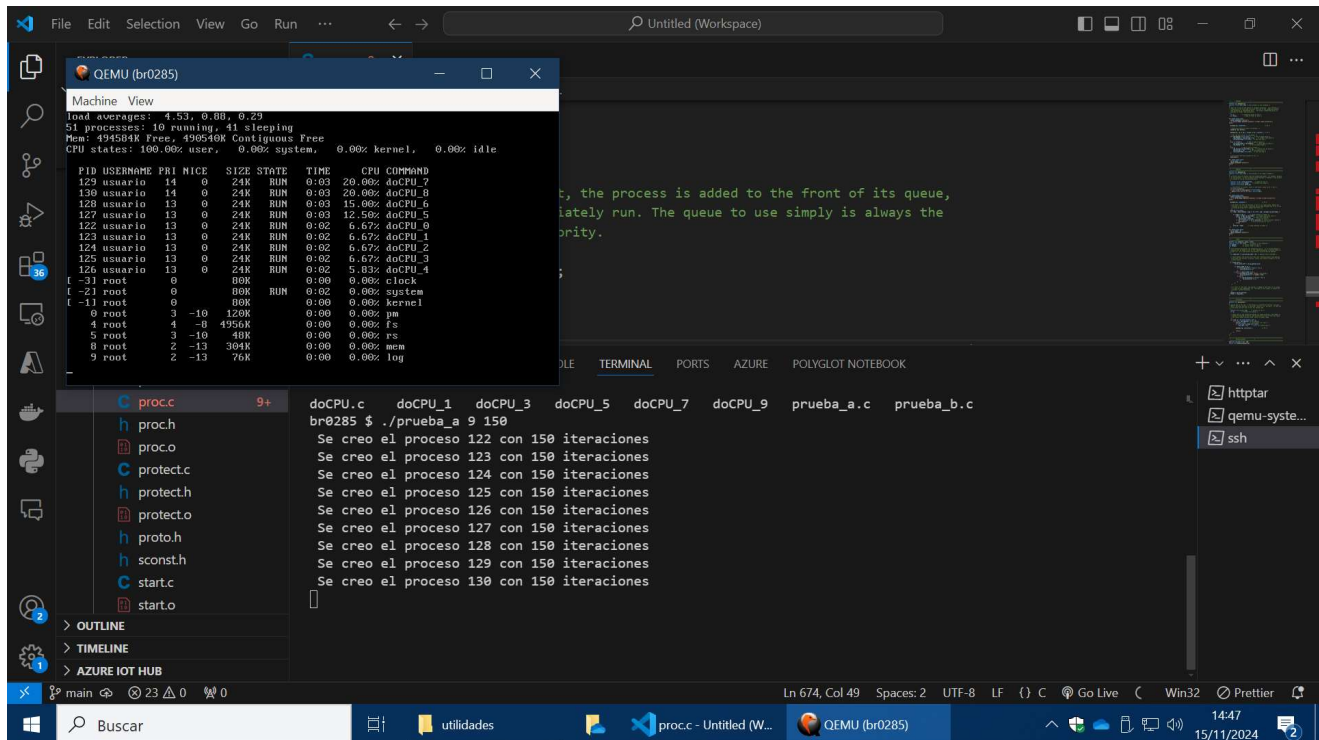
| indOp | op       | PID | pnr | PR | MX | TL | QS | st | p_name | mp_name | call | call name | ds      |
|-------|----------|-----|-----|----|----|----|----|----|--------|---------|------|-----------|---------|
| 0     | SENDREC  | 166 | 94  | 7  | 7  | 4  | 8  | 00 | plot   | plot    | 1    | EXIT      | -> pm 0 |
| 1     | ENQUEUE  | 0   | 0   | 3  | 3  | 32 | 32 | 00 | pm     | pm      |      |           |         |
| 2     | SCHED    | 0   | 0   | 3  | 1  |    |    |    | pm     | pm      |      |           |         |
| 3     | PICKPROC | 0   | 0   |    |    |    |    |    | pm     | pm      |      |           |         |

DoCPU\_0

## DoCPU\_2

| indOp | op       | PID  | pnr | PR | MX | TL | QS | st | p_name  | mp_name | call | call name | ds |
|-------|----------|------|-----|----|----|----|----|----|---------|---------|------|-----------|----|
| 26591 | PICKPROC | 0    | 0   | 3  | 3  | 28 | 32 | 08 | pm      | pm      |      |           |    |
| 26592 | ENQUEUE  | 170  | 11  | 9  | 7  | 6  | 8  | 00 | prueba_ | doCPU_2 |      |           |    |
| 26593 | SCHED    | 170  | 11  | 9  | 1  |    |    |    | prueba_ | doCPU_2 |      |           |    |
| 26594 | PICKPROC | [-2] | -2  |    |    |    |    |    | system  |         |      |           |    |

## 4. Prioridades de doCPU aleatorias

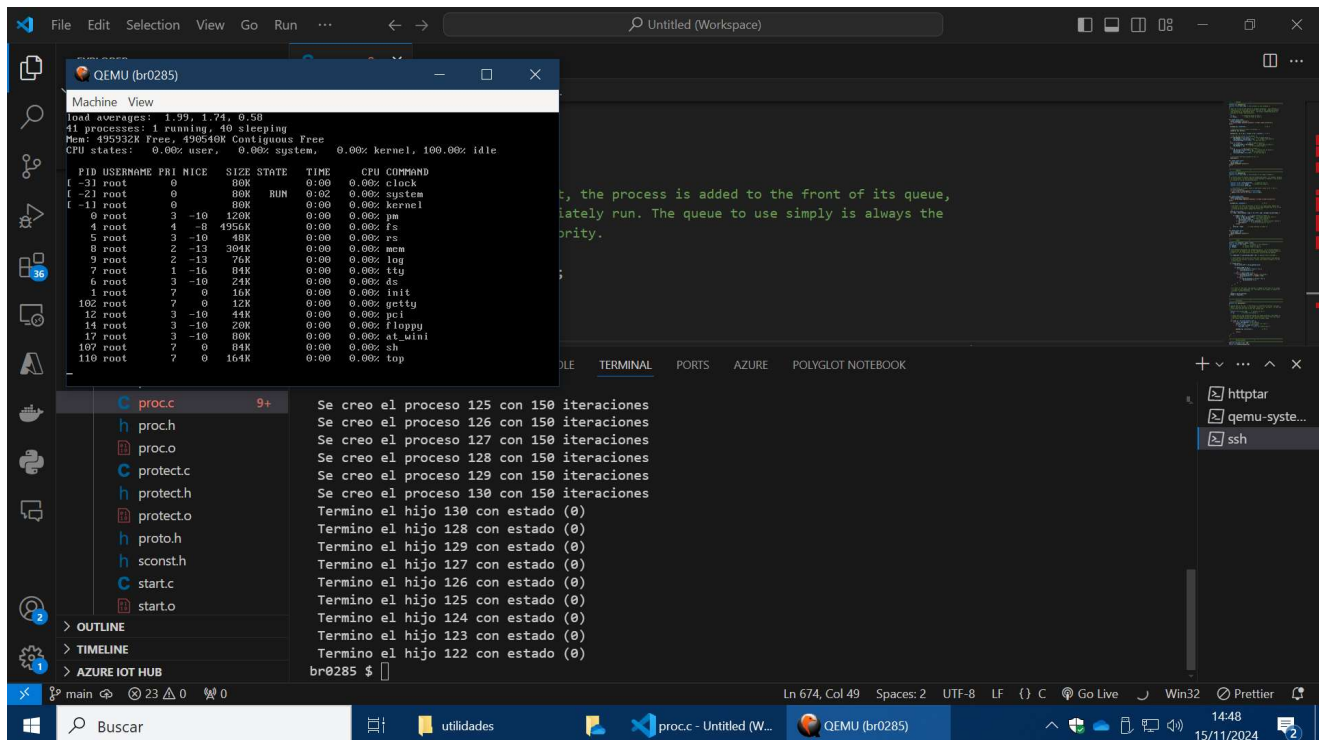


The screenshot shows a QEMU (br0285) window with a 'Machine View' tab. The 'Machine View' displays system statistics and a process list. The process list shows various processes running, including 'doCPU.c' and 'doCPU\_1' through 'doCPU\_9'. The terminal window shows the output of the 'doCPU.c' program, which is creating and scheduling processes with random priorities.

```
Machine View
load averages: 4.53, 0.08, 0.29
51 processes: 10 running, 41 sleeping
Mem: 494504K Free, 496540K Contiguous Free
CPU states: 100.00% user, 0.00% system, 0.00% kernel, 0.00% idle
```

| PID | USERNAME | PRI | NICE | SIZE  | STATE | TIME  | CPU    | COMMAND |
|-----|----------|-----|------|-------|-------|-------|--------|---------|
| 129 | usuario  | 14  | 0    | 24K   | RUN   | 0:03  | 20.00% | doCPU_7 |
| 130 | usuario  | 14  | 0    | 24K   | RUN   | 0:03  | 20.00% | doCPU_8 |
| 128 | usuario  | 13  | 0    | 24K   | RUN   | 0:03  | 15.00% | doCPU_6 |
| 127 | usuario  | 13  | 0    | 24K   | RUN   | 0:03  | 12.50% | doCPU_5 |
| 122 | usuario  | 13  | 0    | 24K   | RUN   | 0:02  | 6.67%  | doCPU_0 |
| 123 | usuario  | 13  | 0    | 24K   | RUN   | 0:02  | 6.67%  | doCPU_1 |
| 124 | usuario  | 13  | 0    | 24K   | RUN   | 0:02  | 6.67%  | doCPU_2 |
| 125 | usuario  | 13  | 0    | 24K   | RUN   | 0:02  | 6.67%  | doCPU_3 |
| 126 | usuario  | 13  | 0    | 24K   | RUN   | 0:02  | 5.83%  | doCPU_4 |
| -31 | root     | 0   | 0    | 80K   | RUN   | 0:00  | 0.00%  | clock   |
| -21 | root     | 0   | 0    | 80K   | RUN   | 0:02  | 0.00%  | system  |
| -11 | root     | 0   | 0    | 80K   | RUN   | 0:00  | 0.00%  | kernel  |
| 0   | root     | 3   | -10  | 120K  | 0:00  | 0.00% | pm     |         |
| 4   | root     | 4   | -8   | 4956K | 0:00  | 0.00% | fs     |         |
| 5   | root     | 3   | -10  | 48K   | 0:00  | 0.00% | rs     |         |
| 8   | root     | 2   | -13  | 304K  | 0:00  | 0.00% | mem    |         |
| 9   | root     | 2   | -13  | 76K   | 0:00  | 0.00% | log    |         |

```
doCPU.c doCPU_1 doCPU_3 doCPU_5 doCPU_7 doCPU_9 prueba_a.c prueba_b.c
br0285 $ ./prueba_a 9 150
Se creo el proceso 122 con 150 iteraciones
Se creo el proceso 123 con 150 iteraciones
Se creo el proceso 124 con 150 iteraciones
Se creo el proceso 125 con 150 iteraciones
Se creo el proceso 126 con 150 iteraciones
Se creo el proceso 127 con 150 iteraciones
Se creo el proceso 128 con 150 iteraciones
Se creo el proceso 129 con 150 iteraciones
Se creo el proceso 130 con 150 iteraciones
```



The screenshot shows a QEMU (br0285) window with a 'Machine View' tab. The 'Machine View' displays system statistics and a process list. The process list shows various processes running, including 'doCPU.c' and 'doCPU\_1' through 'doCPU\_9'. The terminal window shows the output of the 'doCPU.c' program, which is creating and scheduling processes with random priorities.

```
Machine View
load averages: 1.99, 1.74, 0.50
41 processes: 1 running, 40 sleeping
Mem: 495324K Free, 496540K Contiguous Free
CPU states: 0.00% user, 0.00% system, 0.00% kernel, 100.00% idle
```

| PID | USERNAME | PRI | NICE | SIZE  | STATE | TIME  | CPU     | COMMAND |
|-----|----------|-----|------|-------|-------|-------|---------|---------|
| -31 | root     | 0   | 0    | 80K   | RUN   | 0:00  | 0.00%   | clock   |
| -21 | root     | 0   | 0    | 80K   | RUN   | 0:02  | 0.00%   | system  |
| -11 | root     | 0   | 0    | 80K   | RUN   | 0:00  | 0.00%   | kernel  |
| 0   | root     | 3   | -10  | 120K  | 0:00  | 0.00% | pm      |         |
| 4   | root     | 4   | -8   | 4956K | 0:00  | 0.00% | fs      |         |
| 5   | root     | 3   | -10  | 48K   | 0:00  | 0.00% | rs      |         |
| 8   | root     | 2   | -13  | 304K  | 0:00  | 0.00% | mem     |         |
| 9   | root     | 2   | -13  | 76K   | 0:00  | 0.00% | log     |         |
| 7   | root     | 1   | -16  | 84K   | 0:00  | 0.00% | tty     |         |
| 6   | root     | 3   | -10  | 24K   | 0:00  | 0.00% | ds      |         |
| 1   | root     | 7   | 0    | 16K   | 0:00  | 0.00% | init    |         |
| 102 | root     | 7   | 0    | 12K   | 0:00  | 0.00% | getty   |         |
| 12  | root     | 3   | -10  | 44K   | 0:00  | 0.00% | pci     |         |
| 14  | root     | 3   | -10  | 20K   | 0:00  | 0.00% | floppy  |         |
| 17  | root     | 3   | -10  | 80K   | 0:00  | 0.00% | at_wini |         |
| 107 | root     | 7   | 0    | 84K   | 0:00  | 0.00% | sh      |         |
| 110 | root     | 7   | 0    | 164K  | 0:00  | 0.00% | top     |         |

```
Se creo el proceso 125 con 150 iteraciones
Se creo el proceso 126 con 150 iteraciones
Se creo el proceso 127 con 150 iteraciones
Se creo el proceso 128 con 150 iteraciones
Se creo el proceso 129 con 150 iteraciones
Se creo el proceso 130 con 150 iteraciones
Termino el hijo 130 con estado (0)
Termino el hijo 128 con estado (0)
Termino el hijo 129 con estado (0)
Termino el hijo 127 con estado (0)
Termino el hijo 126 con estado (0)
Termino el hijo 125 con estado (0)
Termino el hijo 124 con estado (0)
Termino el hijo 123 con estado (0)
Termino el hijo 122 con estado (0)
br0285 $
```

```
/*=====*/
* sched *
*=====*/
PRIVATE void sched(rp, queue, front)
register struct proc *rp; /* process to be scheduled */
int *queue; /* return: queue to use */
int *front; /* return: front or back */
{
```

```

/* This function determines the scheduling policy. It is called whenever a
 * process must be added to one of the scheduling queues to decide where to
 * insert it. As a side-effect the process' priority may be updated.
 */
int time_left = (rp->p_ticks_left > 0); /* quantum fully consumed */

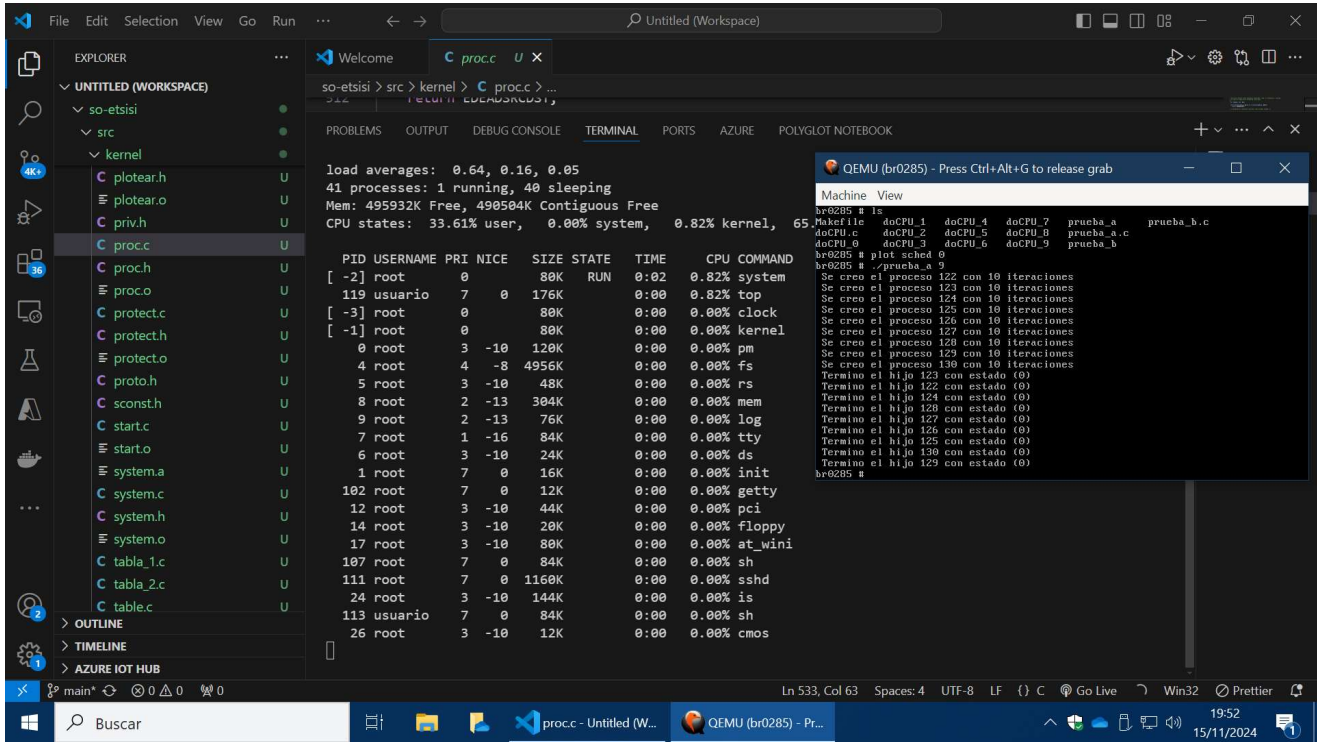
/* Check whether the process has time left. Otherwise give a new quantum
 * and lower the process' priority, unless the process already is in the
 * lowest queue.
 */
if (!time_left) {
    rp->p_ticks_left = rp->p_quantum_size;

    if (plot_sched == 0) {
        if (rp->p_priority < (IDLE_Q - 1)) {
            rp->p_priority += 1;
        }
    } else if (plot_sched == 1) {
        if (strncmp(rp->p_name, "doCPU_", 6) == 0) {
            rp->p_priority = 5 + (rand() % 10);
        } else {
            if (rp->p_priority < (IDLE_Q - 1)) {
                rp->p_priority += 1;
            }
        }
    }
}

/* If there is time left, the process is added to the front of its queue,
 * so that it can immediately run. The queue to use simply is always the
 * process' current priority.
 */
*queue = rp->p_priority;
*front = time_left;
}

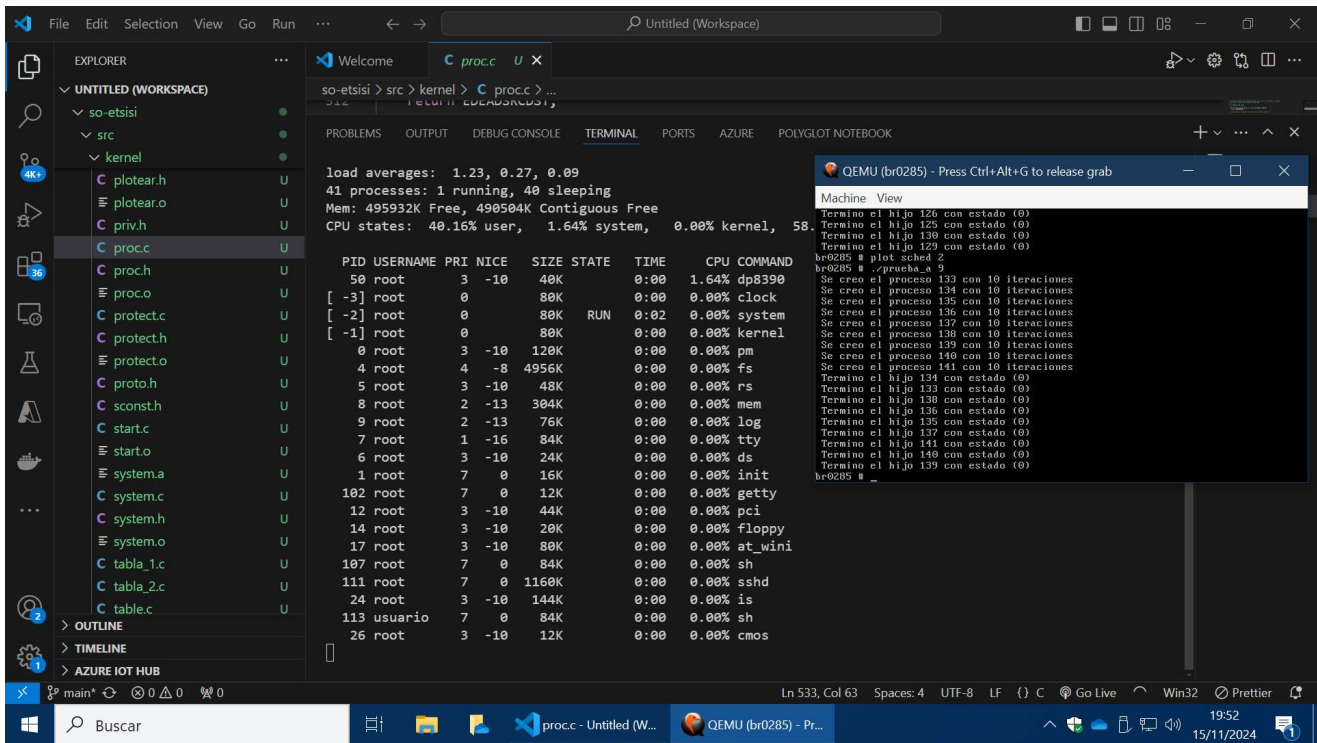
```

# 5. Modificando el encolado en las colas de prioridad de procesos



```
load averages: 0.64, 0.16, 0.05
41 processes: 1 running, 40 sleeping
Mem: 495932K Free, 490504K Contiguous Free
CPU states: 33.61% user, 0.00% system, 0.82% kernel, 65.57% idle
```

| PID   | USERNAME | PRI | NICE | SIZE  | STATE | TIME | CPU   | COMMAND |
|-------|----------|-----|------|-------|-------|------|-------|---------|
| [ -2] | root     | 0   | 0    | 80K   | RUN   | 0:02 | 0.82% | system  |
| 119   | usuario  | 7   | 0    | 176K  | U     | 0:00 | 0.82% | top     |
| [ -3] | root     | 0   | 0    | 80K   | U     | 0:00 | 0.00% | clock   |
| [ -1] | root     | 0   | 0    | 80K   | U     | 0:00 | 0.00% | kernel  |
| 0     | root     | 3   | -10  | 120K  | U     | 0:00 | 0.00% | pm      |
| 4     | root     | 4   | -8   | 4956K | U     | 0:00 | 0.00% | fs      |
| 5     | root     | 3   | -10  | 48K   | U     | 0:00 | 0.00% | rs      |
| 8     | root     | 2   | -13  | 304K  | U     | 0:00 | 0.00% | mem     |
| 9     | root     | 2   | -13  | 76K   | U     | 0:00 | 0.00% | log     |
| 7     | root     | 1   | -16  | 84K   | U     | 0:00 | 0.00% | tty     |
| 6     | root     | 3   | -10  | 24K   | U     | 0:00 | 0.00% | ds      |
| 1     | root     | 7   | 0    | 16K   | U     | 0:00 | 0.00% | init    |
| 102   | root     | 7   | 0    | 12K   | U     | 0:00 | 0.00% | getty   |
| 12    | root     | 3   | -10  | 44K   | U     | 0:00 | 0.00% | pci     |
| 14    | root     | 3   | -10  | 20K   | U     | 0:00 | 0.00% | floppy  |
| 17    | root     | 3   | -10  | 80K   | U     | 0:00 | 0.00% | at_wini |
| 107   | root     | 7   | 0    | 84K   | U     | 0:00 | 0.00% | sh      |
| 111   | root     | 7   | 0    | 1160K | U     | 0:00 | 0.00% | sshd    |
| 24    | root     | 3   | -10  | 144K  | U     | 0:00 | 0.00% | is      |
| 113   | usuario  | 7   | 0    | 84K   | U     | 0:00 | 0.00% | sh      |
| 26    | root     | 3   | -10  | 12K   | U     | 0:00 | 0.00% | cmos    |



```
load averages: 1.23, 0.27, 0.09
41 processes: 1 running, 40 sleeping
Mem: 495932K Free, 490504K Contiguous Free
CPU states: 40.16% user, 1.64% system, 0.00% kernel, 58.20% idle
```

| PID   | USERNAME | PRI | NICE | SIZE  | STATE | TIME | CPU   | COMMAND |
|-------|----------|-----|------|-------|-------|------|-------|---------|
| 50    | root     | 3   | -10  | 40K   | U     | 0:00 | 1.64% | dp8390  |
| [ -3] | root     | 0   | 0    | 80K   | U     | 0:00 | 0.00% | clock   |
| [ -2] | root     | 0   | 0    | 80K   | RUN   | 0:02 | 0.00% | system  |
| [ -1] | root     | 0   | 0    | 80K   | U     | 0:00 | 0.00% | kernel  |
| 0     | root     | 3   | -10  | 120K  | U     | 0:00 | 0.00% | pm      |
| 4     | root     | 4   | -8   | 4956K | U     | 0:00 | 0.00% | fs      |
| 5     | root     | 3   | -10  | 48K   | U     | 0:00 | 0.00% | rs      |
| 8     | root     | 2   | -13  | 304K  | U     | 0:00 | 0.00% | mem     |
| 9     | root     | 2   | -13  | 76K   | U     | 0:00 | 0.00% | log     |
| 7     | root     | 1   | -16  | 84K   | U     | 0:00 | 0.00% | tty     |
| 6     | root     | 3   | -10  | 24K   | U     | 0:00 | 0.00% | ds      |
| 1     | root     | 7   | 0    | 16K   | U     | 0:00 | 0.00% | init    |
| 102   | root     | 7   | 0    | 12K   | U     | 0:00 | 0.00% | getty   |
| 12    | root     | 3   | -10  | 44K   | U     | 0:00 | 0.00% | pci     |
| 14    | root     | 3   | -10  | 20K   | U     | 0:00 | 0.00% | floppy  |
| 17    | root     | 3   | -10  | 80K   | U     | 0:00 | 0.00% | at_wini |
| 107   | root     | 7   | 0    | 84K   | U     | 0:00 | 0.00% | sh      |
| 111   | root     | 7   | 0    | 1160K | U     | 0:00 | 0.00% | sshd    |
| 24    | root     | 3   | -10  | 144K  | U     | 0:00 | 0.00% | is      |
| 113   | usuario  | 7   | 0    | 84K   | U     | 0:00 | 0.00% | sh      |
| 26    | root     | 3   | -10  | 12K   | U     | 0:00 | 0.00% | cmos    |

```
PRIVATE void enqueue(rp)
register struct proc *rp; /* this process is now runnable */
{
/* Add 'rp' to one of the queues of runnable processes. This function is
```

```

* responsible for inserting a process into one of the scheduling queues.
* The mechanism is implemented here. The actual scheduling policy is
* defined in sched() and pick_proc().
*/
int q;                /* scheduling queue to use */
int front;            /* add to front or back */

#ifdef DEBUG_SCHED_CHECK
    check_runqueues("enqueue");
    if (rp->p_ready) kprintf("enqueue() already ready process\n");
#endif

    plotear(rp, opENQUEUE) ;                /* PP */

    /* Determine where to insert to process. */
    sched(rp, &q, &front);

    plotear(rp, (q << 16) | (front << 8) | opSCHED) ; /* PP */

    if (plot_sched == 0) {
        if (rdy_head[q] == NIL_PROC) {
            rdy_head[q] = rdy_tail[q] = rp;
            rp->p_nextready = NIL_PROC;
        } else if (front) {
            rp->p_nextready = rdy_head[q];
            rdy_head[q] = rp;
        } else {
            rdy_tail[q]->p_nextready = rp;
            rdy_tail[q] = rp;
            rp->p_nextready = NIL_PROC;
        }
    } else if (plot_sched == 2) {
        if (rdy_head[q] == NIL_PROC) {
            rdy_head[q] = rdy_tail[q] = rp;
            rp->p_nextready = NIL_PROC;
        } else {
            struct proc *prev = NIL_PROC;
            struct proc *current = rdy_head[q];

            while (current != NIL_PROC && current->p_nr < rp->p_nr) {
                prev = current;
                current = current->p_nextready;
            }

            if (prev == NIL_PROC) {
                rp->p_nextready = rdy_head[q];
                rdy_head[q] = rp;
            } else {
                prev->p_nextready = rp;
                rp->p_nextready = current;
            }
        }
        if (current == NIL_PROC) {
            rdy_tail[q] = rp;
        }
    }
}

```

```
    }  
}  
/* Now select the next process to run. */  
pick_proc();  
  
#if DEBUG_SCHED_CHECK  
    rp->p_ready = 1;  
    check_runqueues("enqueue");  
#endif  
}
```